Task 5: Capture and Analyze Network Traffic Using Wireshark

Nov 11, 2026 · @Shuvo

1. Objective and environment

I captured live traffic on a Kali Linux VM with Wireshark and identified four protocols in a 21-packet capture: DNS, ICMP, ARP and a Dropbox LAN discovery protocol (DB-LSP-DISC).

Objective: capture live network packets and identify basic protocols and traffic types.

Item	Detail
Tool	Wireshark (free, open source)
Operating system	Kali Linux, running as a VM in VMware Workstation
Capture interface	eth0
Kali VM IP address	192.168.182.128
DNS server / gateway	192.168.182.2 (VMware NAT)
Traffic generated	ping www.google.com (6 echo requests)
Capture result	21 packets captured, 0 dropped (0.0%)
Deliverables	Packet capture (.pcap) file and this report

2. Procedure: generating traffic

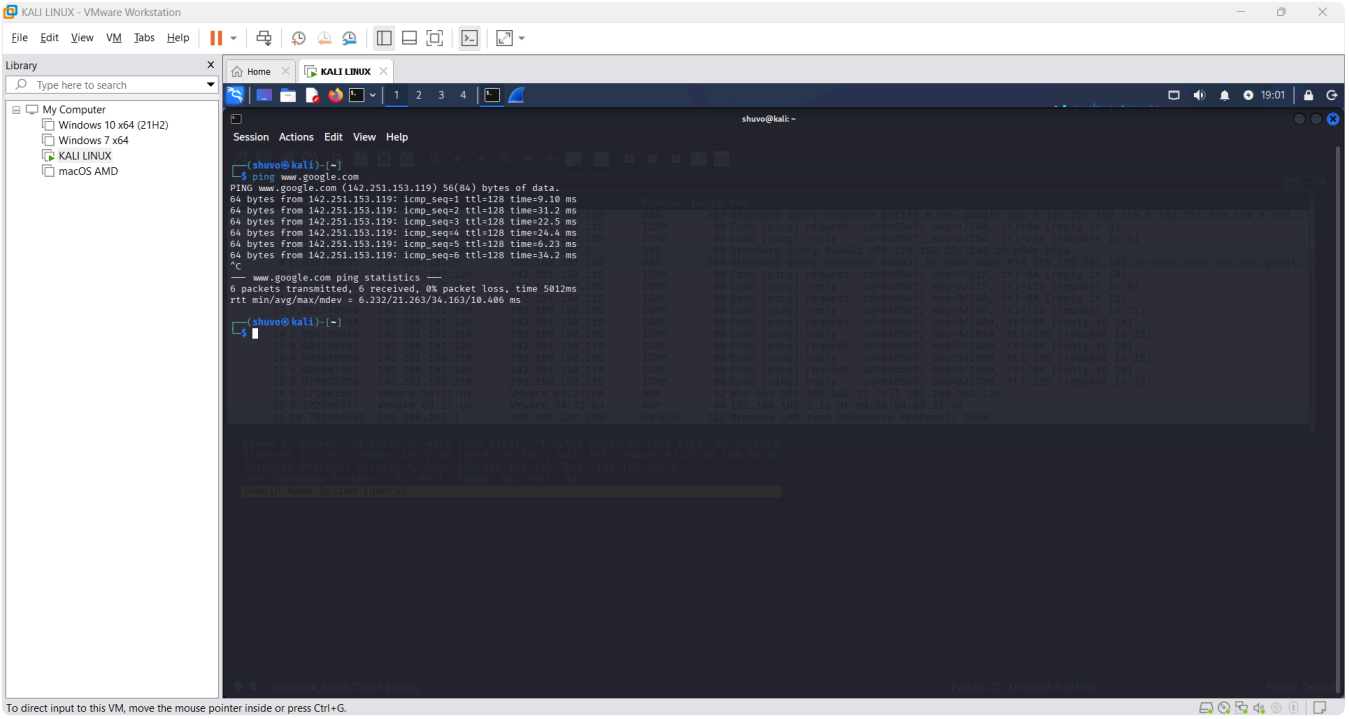
I started a Wireshark capture on eth0, then pinged www.google.com from the Kali terminal; all 6 packets were answered with 0% loss.

1. Install Wireshark (preinstalled on Kali Linux).
2. Start capturing on the active interface, eth0.
3. In a terminal run `ping www.google.com` to generate DNS and ICMP traffic.
4. Stop the ping with Ctrl+C, then stop the capture.
5. Filter the packet list by protocol, identify the protocols, and save the capture as a .pcap file (File > Save As).

The ping output shows the name www.google.com resolved to 142.251.153.119. Each of the 6 echo requests got a reply, with round-trip times between 6.23 ms and 34.2 ms.

Ping statistic	Value
Target IP	142.251.153.119
Packets sent / received	6 / 6
Packet loss	0%
Total time	5012 ms
RTT min / avg / max / mdev	6.232 / 21.263 / 34.163 / 10.406 ms

Screenshot 1: ping www.google.com in the Kali Linux terminal



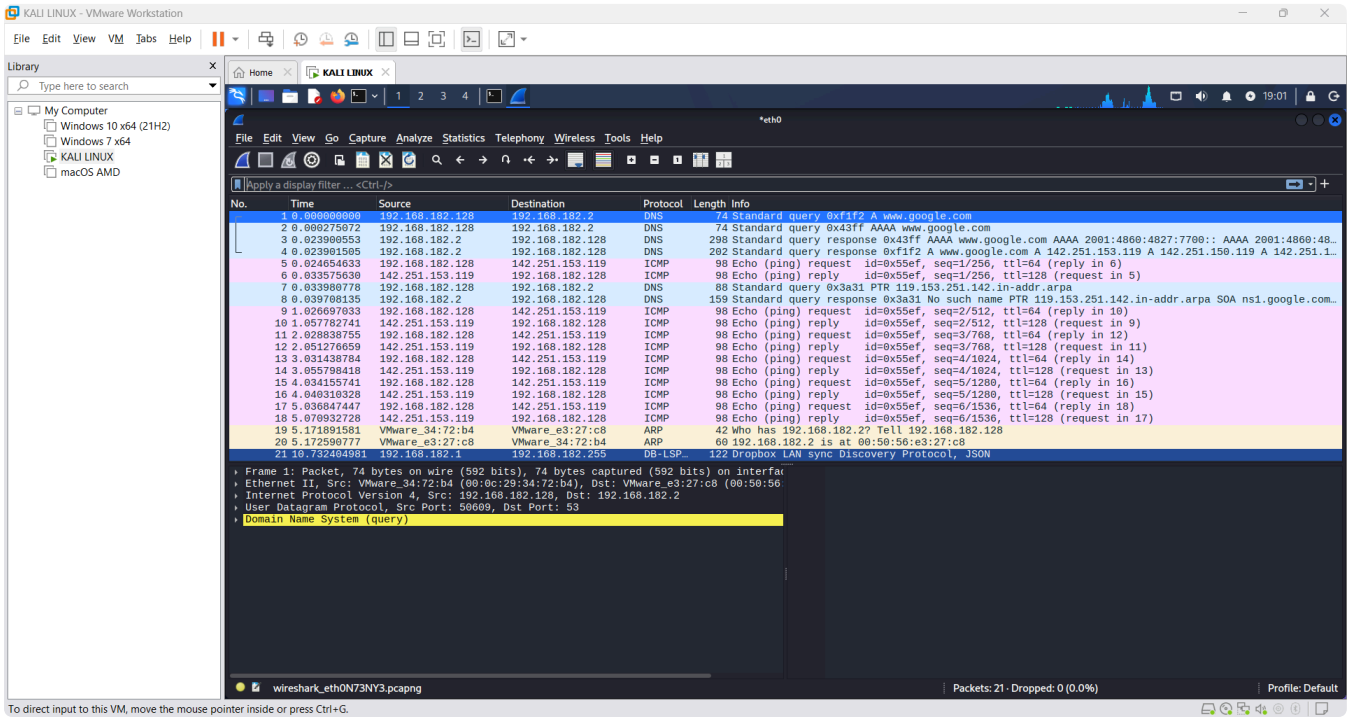
3. Wireshark capture

The capture on eth0 holds 21 packets with none dropped; the packet list shows DNS lookups first, then the ping, then ARP and one Dropbox broadcast.

The capture was saved by Wireshark as `wireshark_eth0N73NY3.pcapng`, and the status bar reads Packets: 21, Dropped: 0 (0.0%), Profile: Default. Frame 1 is selected, and the details pane shows its layers: Frame, Ethernet II, Internet Protocol Version 4, User Datagram Protocol and Domain Name System (query).

The whole capture spans about 10.7 seconds. The ping itself ran for roughly 5 seconds, between packets 5 and 18.

Screenshot 2: Wireshark packet list and Frame 1 details



4. Protocols identified

Four protocols appear in the capture: DNS (6 packets), ICMP (12 packets), ARP (2 packets) and DB-LSP-DISC (1 packet). No TCP or HTTP traffic appeared, because ping uses ICMP and DNS uses UDP.

Protocol	Packets	Layer	What it did here
DNS	1, 2, 3, 4, 7, 8	Application (over UDP port 53)	Resolved www.google.com and ran a reverse lookup
ICMP	5, 6, 9 to 18	Network	Echo request and reply for the 6 pings
ARP	19, 20	Link	Found the MAC address of the gateway 192.168.182.2
DB-LSP-DISC	21	Application (broadcast)	Dropbox LAN sync discovery, JSON payload

DNS

Packet 1 is a standard query for the A record (IPv4) of www.google.com, sent from 192.168.182.128 to 192.168.182.2 using UDP source port 50609 and destination port 53. Packet 2 asks for the AAAA record (IPv6) 0.000275 s later. Packet 3 answers the AAAA

query with IPv6 addresses starting 2001:4860:4827:7700, and packet 4 answers the A query with 142.251.153.119 and 142.251.150.119 among others. Packet 7 is a PTR (reverse) lookup for 119.153.251.142.in-addr.arpa, which ping triggers to print a hostname; packet 8 replied "No such name" with an SOA record from ns1.google.com.

ICMP

Packets 5, 9, 11, 13, 15 and 17 are echo requests from the VM to 142.251.153.119 (TTL 64, 98 bytes, ID 0x55ef, sequence 1 to 6). Packets 6, 10, 12, 14, 16 and 18 are the matching echo replies (TTL 128), and Wireshark links each pair, for example "reply in 6". Requests leave about one second apart, matching the default ping interval, and the sequence numbers 1 to 6 match the six replies in the terminal.

ARP

Packet 19 asks "Who has 192.168.182.2? Tell 192.168.182.128", and packet 20 replies that 192.168.182.2 is at 00:50:56:e3:27:c8. This is the VM refreshing its ARP cache for the gateway so it can send frames to it.

DB-LSP-DISC (Dropbox LAN sync)

Packet 21, at 10.73 s, goes from 192.168.182.1 (the host machine) to the broadcast address 192.168.182.255. It is background traffic from the Dropbox client on the host, not part of the ping test, and shows how a capture can include unrelated devices on the same network.

Frame 1 layer breakdown

Layer	Key fields
Frame	74 bytes on the wire (592 bits)
Ethernet II	Src VMware_34:72:b4 (00:0c:29:34:72:b4), Dst VMware_e3:27:c8 (00:50:56:e3:27:c8)
IPv4	Src 192.168.182.128, Dst 192.168.182.2
UDP	Src port 50609, Dst port 53
DNS	Standard query, type A, www.google.com

Display filters to isolate each protocol

Filter	Shows
`dns`	DNS queries and responses
`icmp`	Ping requests and replies
`arp`	ARP requests and replies
`ip.addr == 142.251.153.119`	All traffic to or from Google's IP

5. Analysis and conclusion

One `ping www.google.com` command produced a full chain of network activity, in this order:

1. **DNS** (packets 1 to 4): the VM asked the gateway/DNS server for Google's IPv4 and IPv6 addresses and got answers in about 24 ms.
2. **ICMP** (packets 5 to 18): the VM pinged the returned IPv4 address six times and received six replies, matching the 0% packet loss in the terminal.
3. **Reverse DNS** (packets 7 and 8): a PTR lookup ran in parallel and returned "No such name", which is harmless.
4. **ARP** (packets 19 and 20): a periodic refresh of the gateway's MAC address.
5. **DB-LSP-DISC** (packet 21): unrelated Dropbox broadcast from another device on the subnet.

The terminal's round-trip times (average 21.3 ms) agree with the request and reply timestamps in Wireshark. The capture shows that Wireshark sees every layer: Ethernet (MAC addresses), IP (addresses), UDP and ICMP (transport and control) and DNS (application).

Conclusion: I captured live traffic, filtered it by protocol, identified four different protocols and explained their packet details. This gave me hands-on practice with packet analysis, the TCP/IP layers and protocol awareness.

6. Interview questions and answers

1. What is Wireshark used for?

Wireshark is a free, open-source packet analyzer that captures network traffic and shows each packet decoded layer by layer. It is used for network troubleshooting, security analysis, protocol learning and checking what an application sends over the network.

2. What is a packet?

A packet is a small unit of data sent over a network. It has a header (source and destination addresses, protocol, length, sequence information) and a payload (the actual data). In this task, Frame 1 was a 74-byte packet carrying a DNS query.

3. How do you filter packets in Wireshark?

There are two kinds. A **capture filter** (BPF syntax, such as `port 53`) limits what is recorded and is set before the capture starts. A **display filter** hides packets in an existing capture and is typed in the filter bar, for example `dns`, `icmp`, `arp`, `tcp.port == 443`, `ip.addr == 142.251.153.119` or `http.request`. Filters can be combined with `&&`, `||` and `!`.

4. What is the difference between TCP and UDP?

Feature	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Acknowledgements and retransmission	No guarantee of delivery
Ordering	Keeps packets in order	No ordering
Speed / overhead	Slower, larger header (20+ bytes)	Faster, 8-byte header
Typical use	HTTP/HTTPS, SSH, email, file transfer	DNS, streaming, VoIP, gaming

In this capture, DNS used UDP (port 53).

5. What is a DNS query packet?

A DNS query packet is a request sent by a client, usually over UDP port 53, asking a DNS server to translate a name into data such as an IP address. Its type says what is wanted: A (IPv4), AAAA (IPv6) or PTR (reverse lookup). In the capture, packet 1 asked for the A record of www.google.com and the server replied in packet 4 with 142.251.153.119 and other addresses.

6. How can packet capture help in troubleshooting?

It shows what is really on the wire. You can find failed DNS lookups, missing replies, high latency, retransmissions, wrong IP or MAC addresses, blocked ports and unexpected traffic. For example, a missing ICMP reply or a "No such name" DNS answer points straight to where a connection breaks.

7. What is a protocol?

A protocol is a set of agreed rules for how devices format, send, receive and acknowledge data. Examples seen here: DNS (name resolution), ICMP (error and diagnostic messages, used by ping), ARP (mapping IP to MAC addresses) and UDP (transport).

8. Can Wireshark decrypt encrypted traffic?

Only in some cases. Wireshark cannot break strong encryption by itself. It can decrypt TLS/HTTPS if you supply the session keys (for example through the SSLKEYLOGFILE setting in the browser or the server's private key for older RSA key exchange), and it can decrypt WPA/WPA2 Wi-Fi traffic if you provide the password and capture the handshake. Without keys, encrypted payloads stay unreadable, though headers and metadata remain visible.